

PUBLIC-SAFE RUNBOOK

MARDUK Public Proxmox Deploy Runbook

A step-by-step guide for the public starter path: run the local proof, create private configuration, review generated plans, and understand what must be proven before a real Proxmox install is called complete.

Short answer

The public repo is useful and testable today. It is not a full public Proxmox installer yet. A real install still needs private config, user-owned secrets, DNS/firewall setup, backup custody, and second-hardware proof.

One physical host = one fate domain

MARDUK recreates managed-cloud control loops on owned hardware. It does not pretend one box is multiple independent failure domains.

Recommended path

public proof -> private config -> generated plans -> real install proof

Plain-English Status

This guide separates three things that are easy to mix up: what the public repo proves today, what a private operator must still provide, and what must pass before anyone should call the public Proxmox install complete.

Layer	Meaning	Current status
Public starter	Safe files, examples, local checks, and disposable proofs.	Usable today.
Private operator setup	Your own topology, custody, accounts, and trust decisions.	Required for any real install.
Full public install package	A new operator can clone the repo and build a real Proxmox platform with user-owned inputs.	Not proven yet.

The rule for this runbook

Do the public proof first. Then create a private config. Then review generated plans. Only then touch real infrastructure.

Mental Model

MARDUK is easiest to follow as a ladder. Each rung proves one smaller thing before the next one gets more dangerous or more private.

Rung	What happens	Boundary
1. Local proof	No private values. Disposable checks.	No real infrastructure changes.
2. Private config	Your topology and custody decisions.	Values stay outside the public repo.
3. Generated plans	Review before real changes.	Generated files stay private.
4. Real install	Terraform, Talos, GitOps, OpenBao, verify.	Only after review and human gates.

Control objective	Mechanism	Proof	Boundary
Keep Git public-safe	Examples and placeholders	Secret scans and public proof	No real secrets in public repo
Avoid manual drift	GitOps write path	Runtime state reconciles from Git	Git does not mean direct cluster writes
Keep secrets out of Git	OpenBao plus External Secrets	Disposable auth and sync proofs	Real values stay private
Recover deliberately	Snapshot, rebuild, verify	Private timed rebuild proof	Not a full public install claim

Step 0 - Know What You Need Before You Start

Goal

This step prevents the most common mistake: treating a public starter repo like it already knows your infrastructure.

Success looks like

- A workstation with Git, shell tools, Docker, and Make.
- A Proxmox host or lab cluster you control for later steps.
- A private place to keep config, generated state, and custody material.

Stop if

- You plan to paste secrets into chat, Git, issues, screenshots, or public logs.
- You do not own the Proxmox, DNS, firewall, backup, or registry accounts.
- You need a one-command real installer today.

Proof to keep

Write down who owns each external gate: Proxmox, DNS/public edge, firewall, vault custody, backup target, observability, and signing identity.

Step 1 - Run The Public-Safe Proof

Goal

Prove the repo is healthy without touching real infrastructure or collecting private values.

Run

```
git clone <public-repo-url> marduk-public
cd marduk-public
make doctor
./deploy-marduk-public.sh public-proof
```

Success looks like

- The doctor checks pass.
- The public-proof wrapper completes.
- The output states the honest boundary: this is not a real Proxmox deploy.

Stop if

- A prerequisite check fails.
- The proof asks for a real secret value.
- The proof tries to change real infrastructure.

Proof to keep

Keep the command name, timestamp, and final PASS line. Do not keep full terminal logs if they include local paths or account details.

What this proves

The public path can run starter validation, config rendering, disposable OpenBao/Kubernetes/External Secrets checks, backup proof, local edge proof, and cleanup checks without private infrastructure.

Step 2 - Create Your Private Operational Workspace

Goal

Move from public examples to your own private configuration. This is where real topology belongs.

Run from the public repo checkout

```
PRIVATE_WS=<private-operational-workspace>
mkdir -p "$PRIVATE_WS/generated/terraform" "$PRIVATE_WS/generated/openbao-bootstrap"
cp starter/config/marduk.env.example "$PRIVATE_WS/marduk.env"
# Replace every placeholder with your own non-secret topology.
./deploy-marduk-public.sh verify-config "$PRIVATE_WS/marduk.env"
```

Success looks like

- The private config validates with no placeholders.
- Topology values match your Proxmox, network, GitOps, registry, backup, and observability design.
- The file contains configuration references, not secret values.

Stop if

- Any placeholder remains.
- The config contains passwords, private keys, unseal material, or account credentials.
- You are editing secrets inside the public repo.

Proof to keep

Keep the doctor PASS result and a redacted config checklist. Do not publish the private config file.

Step 3 - Render Plans Before Touching Infrastructure

Goal

Generate the files and ceremony plan, then review them like a change request before any real apply/init step.

Run from the public repo checkout

```
WS="$PRIVATE_WS"
./deploy-marduk-public.sh plan "$WS/marduk.env"
./deploy-marduk-public.sh render-terraform \
"$WS/marduk.env" \
"$WS/generated/terraform/terraform.tfvars"
./deploy-marduk-public.sh openbao-plan "$WS/marduk.env"
./deploy-marduk-public.sh render-openbao \
"$WS/marduk.env" \
"$WS/generated/openbao-bootstrap"
./deploy-marduk-public.sh openbao-first-install-dry-run \
"$WS/marduk.env"
```

Success looks like

- The plan matches the intended topology.
- Terraform variables are generated for your private workspace only.
- The OpenBao bundle contains policies and role payloads, not secret values.

Stop if

- The plan points at the wrong host, bridge, storage, network, repo, registry, or backup target.
- Generated files contain secrets or real credentials.
- The dry run is skipped before a live ceremony.

Proof to keep

Keep the reviewed plan summary and the fact that the dry run passed. Store generated files only in the private workspace.

Step 4 - Build Real Infrastructure

Important boundary

This page is the real install sequence, not a completed public installer. Use it as the order of operations until a fully packaged public deploy wrapper exists and is proven on independent hardware.

Order	Action	Success signal
1	Build the VM substrate with Terraform from reviewed private variables.	Only intended VM resources are created or changed.
2	Bootstrap Talos and Kubernetes using fresh operator-owned secrets.	Nodes join and Kubernetes control plane is healthy.
3	Seed GitOps controllers and platform apps from Git.	Applications reconcile from Git, not from hand edits.
4	Run OpenBao first-install or restore through the documented human gate.	Vault is unsealed and ready without exposing custody material.
5	Connect External Secrets to materialize runtime values.	Target runtime secrets sync without putting values in Git.
6	Verify admission, route, observability, backup, and recovery checks.	Final platform verification is green.

Stop if

Any plan wants to destroy unrelated resources, any custody material is missing, or any secret appears in a generated file, chat, issue, screenshot, or public log.

Human Gates

Some platform steps should require a human. The point is not to automate trust away. The point is to make automation prove the human-owned decision worked.

Gate	Human owns	Automation proves
Proxmox authority	Endpoint, node, storage, bridge, and state ownership.	Only intended resources change.
DNS and public edge	Account ownership, names, routes, and scope.	Expected route succeeds; wrong route fails.
Firewall	Rule intent, order, source, and destination.	Packet path reaches only the intended target.
Vault custody	Unseal material and recovery process.	Vault recovers and secret sync resumes.
Backup target	Receiver account, host trust, and retention.	Snapshot lands outside the cluster with expected size/hash.
Signing identity	Private signing key custody and rotation.	Signed image admitted; unsigned image denied.

Plain rule

Good automation verifies gates. Bad automation stores secret values or marks gates complete without testing them.

Secrets And Runtime Values

The public repo should never carry the values that make an environment yours. Git can carry references, policies, and examples. Runtime values belong in the secret manager and private custody process.

Git may contain	Git must not contain
Example config with placeholders. Policy templates and role payloads. References to expected runtime values. Validation scripts and proof wrappers.	Passwords or private keys. Vault unseal material. Account credentials. Registry, DNS, backup, or signing secrets.

Runtime model

OpenBao carries values. External Secrets materializes runtime state. Git carries references and policy.

Proof to keep

Disposable proofs can validate vault auth and External Secrets sync without printing real values. Real values must be verified privately and redacted before any public evidence is shared.

Final Verification Checklist

Use this checklist after a real private install or rebuild. A step is not complete just because a command returned zero; it is complete when the expected control behavior is observed.

Area	Check	Pass condition
Substrate	Terraform state and VM inventory	Only intended nodes exist; no unexpected drift.
Cluster	Kubernetes readiness	All intended nodes are Ready and core system pods are healthy.
GitOps	Application reconciliation	Runtime state matches Git after sync.
Drift	Manual edit test	Unauthorized manual change is reverted quickly.
Supply chain	Admission policy	Signed digest admitted; unsigned or tampered image denied.
Secrets	Vault and External Secrets	Vault unsealed; runtime sync is healthy.
Routing	Public or lab route	Expected route succeeds; unknown route fails.
Observability	Targets and alerts	Metrics evaluate and test alerts fire.
Recovery	Backup and restore	Snapshot can be restored and verified.

Troubleshooting Stop Conditions

These are hard stops. Fix the cause before continuing. Continuing past one of these usually creates worse evidence, not a better install.

Stop condition	What it usually means	Next move
Doctor fails	Tooling or config shape is wrong.	Fix prerequisites or config before any plan/render step.
Generated plan surprises you	The config does not match the intended topology.	Change config, rerender, and review again.
Any secret appears in a file or log	Custody boundary failed.	Stop, rotate if needed, remove exposure, rerun scans.
Terraform wants unrelated changes	State or targeting is wrong.	Do not apply. Inspect state and variables.
Vault cannot be unsealed	Custody material is missing or wrong.	Stop the deploy and recover custody first.
GitOps does not converge	Runtime is not matching Git.	Fix Git or controller health; avoid manual cluster edits.
Admission allows unsigned images	Supply-chain gate is not enforcing.	Treat as a release blocker.

Evidence To Capture

Evidence should prove the control without leaking the environment. Keep it small, structured, and boring.

Good evidence	Do not capture
Command name and timestamp. Final PASS line or summarized result. Before/after status with private values removed. What failed first and how it was fixed. Hash/size of backup artifacts when safe.	Full terminal prompts. Local machine paths. Account menus or private repo details. Hostnames, domains, IPs, or operational topology. Secrets, credentials, or custody material.

Posting rule

For public posts, show the claim, the proof type, and the boundary. Do not show the raw operating environment.

What This Runbook Lets You Claim

Claim	Safe wording
Public repo	A public starter with local proof wrapper and disposable validation paths.
Private implementation	A privately proven implementation exists with timed rebuild and recovery evidence.
Secrets	Git stores references and policy; runtime values stay private.
Supply chain	Admission can deny unsigned, wrong-key, or tampered images when configured.
Recovery	Recovery is timed and verified privately, not assumed.
Boundary	A single physical host is one fate domain; simulated nodes do not equal multi-site HA.

Do not claim	Why
Completed full public Proxmox installer	The full public install path has not been proven on independent hardware yet.
Multi-host HA	The current proof boundary is one physical host unless proven elsewhere.
Zero-risk operations	Human custody, external accounts, and recovery still need discipline.
Production replacement cloud	The project proves platform control patterns, not a universal replacement for managed cloud.

Bottom line

MARDUK is a proof-driven platform pattern: control loops first, evidence second, claims last.